Week 15

Name: AKSHAY P
Mobile:

Data Structure Workouts
1. Learn the concepts of Tree. Complete at least three sample workouts. 2. Learn the concepts of Binary Search Tree. Complete at least three sample workouts. Example: a. Create a Binary Search Tree with insertion, contains, delete, three traversals (postorder, preorder, in order). b. Find the closest value to a given number in a Tree. c. Validate whether a given tree is BST or not. 3. Learn the concepts of Heap. Complete at least three sample workouts. Example: a. Create a min heap & max heap with build, insert, remove. 4. Learn the concept of Heap sort. Complete at least three sample workouts 5. Learn the concepts of Trie. Complete at least 3 sample workouts. 6. Learn the concepts of Graph. Complete at least three sample workouts. 7. Learn the concepts of Graph traversals (BFS, DFS). 8. Do at least 3 problems each for every structure from any competitive coding websites 9. Learn about the applications of all structures you covered this week
A tree is a hierarchical data structure composed of nodes connected by edges. It consists of a root node, which is the topmost node, and each node has zero or more child nodes. Nodes with no children are called leaves, while internal nodes have at least one child. The relationship between nodes creates a branching structure, resembling an inverted tree. *Link to the folder containing code and screenshot of the output*
A Binary Search Tree (BST) is a binary tree data structure where each node has at most two child nodes, typically referred to as the left and right child. The key property of a BST is

Data Structure Workouts
that the value of each node in the left subtree is less than or equal to the node's value, and the value of each node in the right subtree is greater than the node's value. This property allows for efficient search, insertion, and deletion operations. *Link to the folder containing code and screenshot of the output*
A heap is a specialized tree-based data structure commonly used to implement priority queues and sorting algorithms. In a min heap, each node's value is less than or equal to its children, making the smallest element the root. A max heap, conversely, has each node greater than or equal to its children, placing the largest element at the root. Basic operations include building the heap, inserting elements, and removing the minimum or maximum element, depending on the heap type. A heap's implementation involves heapifying to maintain the heap property during insertions and removals, typically using arrays or linked structures. The understanding of these operations lays the groundwork for employing heaps in various computational tasks. *Link to the folder containing code and screenshot of the output*
Heap Sort is a comparison-based sorting algorithm that uses a binary heap data structure to build a max-heap or min-heap. The heap is a specialized tree-based structure that satisfies the heap property – for a max-heap, the value of each node is less than or equal to the values of its children, and for a min-heap, it is the opposite. The algorithm works by repeatedly removing the root of the heap and inserting it into the sorted array. Heap Sort has a time complexity of O(n log n) and is an in-place sorting algorithm. *Link to the folder containing code and screenshot of the output*
A Trie, or Prefix Tree, is a tree-like data structure that is used for efficiently storing and searching a dynamic set of strings or keys. Each node in the Trie represents a character,

Data Structure Workouts
and the path from the root to a particular node forms a string. Tries are particularly useful for tasks like autocomplete systems, spell checkers, and IP routing tables. *Link to the folder containing code and screenshot of the output*
A graph is a data structure that consists of a set of nodes or vertices connected by edges. There are various types of graphs, including directed and undirected graphs, weighted and unweighted graphs, and cyclic and acyclic graphs. Nodes represent entities, while edges depict relationships between these entities. *Link to the folder containing code and screenshot of the output*
Breadth-First Search (BFS): *BFS explores a graph level by level, starting from a source node. It uses a queue to visit all neighbors before moving on to the next level. Widely used for finding the shortest path in unweighted graphs.* *Depth-First Search (DFS):* *DFS explores as far as possible along one branch before backtracking. It uses a stack or recursion to manage exploration. Commonly applied in tasks like topological sorting, cycle detection, and maze solving. Not always optimal for finding the shortest path.* *Link to the folder containing code and screenshot of the output*
"Looking back at the past week, I want to acknowledge the effort I put into diving into graph, tree, DFS, and BFS concepts. It feels good to see myself actively engaging with these topics, demonstrating a proactive approach to understanding complex data structures and algorithms. The journey of grasping these concepts is ongoing, but I've

Data Structure Workouts
made solid progress. I'm committed to sustaining this momentum and continuing to explore the practical applications of these concepts to solidify my understanding. If *Link to the folder containing code and screenshot of the output*